PACKAGE 11 / VOLUME 10

Graduate gaming hardware and research thesis

Student handbook · Modules 46–50

20 developed lessons · 60 quiz questions

Architecture, VLSI, verification, controlled evaluation and the engineering discipline of a defensible thesis.

September 28, 2026
Graduate self-study specialization

Contents

The purpose of this volume . . . 4

Program continuity . . . 4

How to read models honestly . . . 4

Assessment and study pattern . . . 4

Volume map . . . 5

Source notation . . . 5

Module 46 — Advanced computer architecture for gaming workloads . . . 6

Lesson 46.1 — Cache coherence, memory consistency and synchronization . . . 6

Lesson 46.2 — Out-of-order execution, speculation and precise state . . . 8

Lesson 46.3 — Architecture simulation, traces and model validation . . . 10

Lesson 46.4 — Memory systems, chiplets and multi-objective design . . . 12

Module 47 — VLSI, timing and physical implementation . . . 14

Lesson 47.1 — CMOS logic, transistor sizing and delay models . . . 14

Lesson 47.2 — Static timing analysis, clocks and timing exceptions . . . 16

Lesson 47.3 — Synthesis, placement, clock trees, routing and physical checks . . . 18

Lesson 47.4 — Power, area, process corners and design-space closure . . . 20

Module 48 — Verification, reliability and hardware trust boundaries . . . 22

Lesson 48.1 — Formal properties, abstraction and assume-guarantee reasoning . . . 22

Lesson 48.2 — Clock/reset crossings and asynchronous FIFO verification . . . 24

Lesson 48.3 — Fault models, ECC, testability and recovery . . . 26

Lesson 48.4 — Hardware security, thermal limits and assurance review . . . 28

Module 49 — Master's thesis: proposal, implementation and evaluation 30

Lesson 49.1 — Question selection, literature map and thesis scope 30

Lesson 49.2 — Proposal review, baselines and risk-reduction milestones 32

Lesson 49.3 — Implementation, ablations and controlled evaluation 34

Lesson 49.4 — Thesis writing, reproducibility and contribution boundaries 36

Module 50 — Thesis defense, professional practice and continued development 38

Lesson 50.1 — Oral defense and independent artifact reproduction 38

Lesson 50.2 — Mature-codebase integration, review and maintainable contributions 40

Lesson 50.3 — Portfolio, technical interviews and role-specific evidence 42

Lesson 50.4 — Long-term learning, maintenance and final handoff 44

Package 11 · Volume 10 · Computer Science and Computer Engineering for Games
September 28, 2026 · Master's-level self-study specialization · Modules 46–50

The purpose of this volume

A fast component is not automatically a good architecture, a passing simulation is not automatically a proof, and a plausible research story is not automatically supported by evidence. This volume develops the ability to move carefully between those claims. Its recurring artifacts are a cache model, a small queue, timing constraints, fault campaigns and a versioned research record. They connect game-engine workloads with the hardware and engineering process beneath them.

Each lesson contains developed instruction, an original worked example or worked argument, a practical assignment and three quiz questions. The lab workbook provides detailed steps and assessment requirements. The answer key is separate. Read the references where the lesson makes a specification-dependent claim, but do not substitute reading for performing the lab.

Program continuity

The existing full-program manifest defines this as Volume 10, "Graduate Gaming Hardware and Research Thesis," with twenty lessons and sixty quiz questions. Package 11 is this expansion bundle; it is not an eleventh academic volume. Every original lesson ID, title, learning outcome, practical assignment, reading assignment and module project is preserved in `metadata/original_volume10_manifest.json` and the structured course data.

The estimated 810–1,380 hours includes the 400–800-hour primary thesis. Gate J is the culminating review of overlapping work in Modules 49–50, not a demand to invent both a game engine and a GPU. Prior labs may become thesis baselines only when their scope and evidence are suitable. The volume is a self-study curriculum, not an accredited qualification.

How to read models honestly

A model starts with a contract: inputs, state, transitions, outputs and exclusions. A derived value is meaningful only within that contract. The package's read-only cache has no prefetcher or memory-level parallelism; its scheduler has no speculative branch recovery; its mailbox has no analog metastability; and its PPA values are synthetic. Each is useful precisely because its boundary is visible.

Separate four kinds of evidence: a hand derivation; a finite executable test or exhaustive finite model; a completed property proof with explicit assumptions; and a measurement from a specified physical or production system. These tiers answer different questions. Improving one tier does not silently promote it into another.

Assessment and study pattern

Attempt each three-question quiz after the lesson without the answer key. Explain the causal reasoning, not just a keyword. For the lab, predict one result, implement or change something meaningful, test a boundary and deliberately expose one failure. Compare with the instructor guide only after recording your first attempt. The local review policy is two correct quiz answers plus a repaired explanation of the third, and eight of ten lab points with all mandatory conditions satisfied.

Take the module project review before treating the module as a completed prerequisite. The thesis requires a reviewer who can challenge the technical argument and independently run the central experiment. No generated reference output here substitutes for that work.

Volume map

Module	Focus	Main evidence
46	Advanced architecture	Coherence/litmus reasoning, precise state, a validated architecture comparison.
47	VLSI and physical implementation	Timing contracts, a small authorized physical flow and qualified PPA evidence.
48	Verification, reliability and trust	Properties, CDC/reset reasoning, fault injection and an assurance dossier.
49	Thesis execution	Reviewed proposal, running baseline, controlled evaluation and a reproducible argument.
50	Defense and handoff	Independent reproduction, maintainable integration, honest portfolio and continued learning.

Source notation

[S01]–[S20] identify the primary-source directory in the Math and Reference guide. R-numbered assignments retain the original program's reference identifiers; a concordance is included in that guide. A source is assigned reading, not a bundled textbook. Live documentation should be pinned to a release, commit or saved dated snapshot before a research claim depends on it.

Module 46

Advanced computer architecture for gaming workloads

Prerequisites: Modules 34, 37-40 and 41-42.

Module project: Perform an architecture study using a pinned simulator or a carefully bounded analytical model. Compare at least three configurations on representative and adverse game-related workloads, and validate the model against a known reference where possible.

The detailed project acceptance record is in the Lab Workbook. Complete all four lessons before the integrated review.

Lesson 46.1

Cache coherence, memory consistency and synchronization

Learning outcome: Separate agreement about a location from ordering across locations.

Start with three different contracts

A game-engine worker publishes a completed visibility list; another worker consumes it. There are three distinct questions: who may own a cached copy, which memory operations may be observed in which order, and whether the source program is legal. Cache coherence addresses agreement and ownership for a location. A memory-consistency model constrains observations across operations. A language memory model governs the program that a compiler is allowed to transform. Correctness at one layer does not automatically establish correctness at the other two.

Use a two-core, one-cache-line model before a large simulator. Each private cache is Invalid, Shared or Modified. Invalid means the line cannot supply a hit. Shared means a clean copy may coexist with other readers. Modified means this cache holds the writable, authoritative copy; memory can be stale. The invariant is not merely “at most one M.” If one cache is M, every other cache must be I. If two caches are S, their values must agree with each other and with memory in this deliberately simple write-back model.

Derive transitions, not slogans

Initially both caches are I and memory contains zero. Core A reads: A becomes S with zero. B reads: both are S. A writes seven: B is invalidated and A becomes M with seven. B then reads: the owner supplies or writes back seven, after which both may become S. B writes nine: A becomes I and B becomes M. At each step, compare a subsequent read against a single-copy reference register.

The reference code makes each request an atomic transition. Real protocols cannot make messages instantaneous. While A waits for invalidation acknowledgments, its state is transient: it has requested ownership but cannot yet promise globally exclusive write permission. A race worth drawing is B's read arriving between A's ownership request and the last acknowledgment. Your extension must say whether the read is queued, retried, forwarded or serialized. Merely adding an “M” flag early can violate the invariant even though every completed sequential trace looks correct. The gem5 Ruby documentation is a route to richer protocol models, not validation of this toy model. [S03]

Publication in C++ is a separate argument

For a one-shot publication, the producer writes a non-atomic payload and performs a release store to an atomic ready flag. The consumer waits until an acquire load reads the value from that release, then reads the payload. The payload write is sequenced before release; the release synchronizes with the observing acquire; the payload read follows acquire. This establishes the needed happens-before relation, provided the producer does not modify the payload again while the consumer reads it. The exact synchronization rule comes from the C++ atomic-ordering specification. [S01]

Replacing both flag operations with relaxed atomics removes that publication argument. If the payload remains non-atomic, unsynchronized conflicting accesses can make the program undefined rather than a useful hardware litmus test. Use atomic payload and flag locations when exploring weak outcomes without undefined behavior. Conversely, a release/acquire pair does not necessarily create a total order among all threads and all locations. State the precise reads-from edge you require.

Worked reasoning: store buffering

Let x and y be separate atomics initially zero. Thread A stores x=1 and then loads y into rA. Thread B stores y=1 and then loads x into rB. Under sequential consistency, rA=rB=0 would require each load to precede the other thread's store, forming a cycle with the two program-order edges. Thus that outcome cannot occur. With relaxed operations, do not import that total-order proof; an execution model may permit both loads to observe zero. A finite hardware test that never observes zero/zero does not prove it forbidden. The RISC-V RVWMO specification illustrates why allowed executions must be defined by explicit ordering rules rather than a picture of caches. [S02]

Apply this to an engine

A false-sharing problem occurs when different workers repeatedly modify different words that occupy one coherence unit. The logical data are independent, but ownership is not. Padding can trade footprint for fewer transfers; batching can reduce publication frequency. Neither repair excuses a data race. Measure the affected phase and record line-size assumptions rather than declaring a padding constant universally correct.

Before the lab: predict the cache states and read values for R(A), R(B), W(A,7), R(B), W(B,9), R(A). Then annotate the publication program with language-level edges. Keep these two proofs on separate pages: that separation is the central skill.

Practical assignment

Model a small coherence protocol and evaluate message-passing litmus tests under stated language and hardware models. Pass: ownership invariants hold, transient races are considered and coherence is not confused with data-race freedom.

Use Lab 46.1 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R48: multiprocessors, coherence and consistency; R35: selected memory-ordering specification; R11: language-level concurrency.

Expansion sources: S01, S02, S03. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

46.1.Q1 After A writes seven while B holds a shared copy, what ownership condition must hold before A is considered Modified?

46.1.Q2 Why is a coherent cache system insufficient justification for a C++ producer using a relaxed ready flag with a non-atomic payload?

46.1.Q3 Under sequential consistency, why is the zero/zero store-buffering outcome impossible, and why is “not observed” weaker than “forbidden”?

Lesson 46.2

Out-of-order execution, speculation and precise state

Learning outcome: Increase instruction overlap without changing architectural outcomes.

Architectural order versus execution order

A processor can start a later independent operation while an earlier operation waits for data. That is useful for game workloads containing independent entity updates or address calculations. It is not permission to change the program's visible result. Separate fetching, renaming, issuing, completing and retiring. Issue consumes execution resources and ready operands. Completion produces a speculative result. Retirement makes an instruction's effect architectural in program order.

True read-after-write dependencies carry values and cannot be removed by renaming. Write-after-read and write-after-write dependencies arise when different logical values share one architectural register name. Assigning distinct physical destinations can remove those name dependencies, but requires a map and a reclamation policy. A reorder buffer tracks age, readiness and exception status. Memory operations introduce additional hazards: a load cannot blindly overtake an older store whose address might overlap without a mechanism for checking and repairing a wrong speculation.

Build a scheduling model with explicit boundaries

The bundled model takes a directed acyclic list of instructions. Each instruction produces an integer equal to its immediate plus its predecessors' results and has a fixed positive latency. At cycle t, previously issued operations whose finish time is t complete; at most one oldest completed instruction retires; then up to W ready instructions issue. An instruction issued at t with latency L becomes available at t+L. The out-of-order policy searches an age-bounded window. The in-order policy stops at the oldest unissued instruction if it is not ready.

These conventions eliminate off-by-one ambiguity. They also expose the limitations: this is not a real ISA core, has no branch predictor, assumes unlimited fully pipelined functional-unit capacity apart from issue width, and has no cache, memory aliasing or physical-register model. A larger window in this model costs no area or energy. Consequently, a scheduling improvement is an overlap result, not a prediction of a commercial CPU's speedup. gem5's O3CPU documentation identifies a richer rename/issue/commit model for a subsequent study. [S04]

Worked schedule

Use four instructions: A has latency four and immediate two; B depends on A, has latency one and immediate three; C is independent with latency one and immediate five; D depends on C, has latency one and immediate seven. Use issue width one, retirement width one and a window of four.

In-order issue starts A at 0, waits at B through cycles 1–3, starts B at 4, C at 5 and D at 6. The final retirement is at cycle 7. Out-of-order issue starts A at 0, C at 1, D at 2 and B at 4. Although C and D finish early, they wait behind B at retirement. Retirement is A at 4, B at 5, C at 6 and D at 7. Both finish at 7: more overlap did not improve this short retirement-limited case. That is a valuable negative result, not a failed example.

Now append a long dependency chain after D or use a wider retirement policy in a separately modified model. Predict which bottleneck moves before running it. Compare complete architectural values, not just cycle counts: A=2, B=5, C=5, D=12. An engine optimization that produces the right total time but wrong visible state is invalid.

Precise exceptions

Suppose B faults while C and D have already executed. A precise architectural boundary contains all effects older than B and no effects from B or younger instructions. The model may record C and D as completed, but must not place their results in the committed map. Stores are especially sensitive because writing external memory early can make rollback impossible. Distinguish “computed a value” from “made an irrevocable effect.” MIT's architecture materials provide the fuller context of speculation and exception mechanisms. [S05]

Diagnose limits using counterexamples

A serial pointer chain exposes little independent work regardless of window size. A wide set of independent operations can instead expose issue bandwidth. A long oldest instruction can fill the window or block retirement. A missing branch model means the simulation cannot justify claims about branch-heavy game scripts. A fixed memory latency means it cannot justify a prefetching claim. Graduate-quality interpretation states which omitted feature could reverse the conclusion and designs a sensitivity test around that omission.

Before the lab: draw issue, finish and retirement rows separately. Reproduce the equal-final-time example and explain why early completion did not reduce final retirement time.

Practical assignment

Use a simulator or detailed scheduling model to compare in-order and out-of-order execution on independent and dependent instruction streams. Pass: architectural results agree and bottleneck explanations include true dependencies and resource limits.

Use Lab 46.2 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R48: superscalar, out-of-order, prediction and exception handling; R38: CPU models and configuration.

Expansion sources: S04, S05. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

46.2.Q1 Which dependencies can register renaming remove, and which dependency must remain?

46.2.Q2 In the worked four-instruction example, why can earlier completion of C and D fail to reduce total retirement time?

46.2.Q3 If B faults after C has completed, which results may appear in precise architectural state?

Lesson 46.3

Architecture simulation, traces and model validation

Learning outcome: Use simulation as evidence with a stated fidelity boundary.

Ask what the model is supposed to decide

A simulator is an executable set of assumptions. Choose its fidelity from the research question. A cache-capacity study may need accurate address reuse but not a detailed branch predictor. A synchronization-tail-latency study may depend on coherence messages, contention, scheduling and operating-system behavior. Start with a decision statement: “At this fixed access stream and line size, does higher associativity remove conflict misses?” That is much narrower and more defensible than “Which CPU is best for games?”

Separate verification, calibration and validation. Verification asks whether the code implements its mathematical specification. Calibration selects parameters using designated evidence. Validation asks whether the calibrated model predicts held-out behavior within its stated purpose. Tuning a miss penalty to match one workload and then presenting that workload as independent validation is circular. Keep a calibration set, a validation set and a record of the features neither covers.

Derive the reference cache

For byte address a and line size B, the block is floor(a/B). With S sets, set index is block modulo S and tag is floor(block/S). A W-way set retains W tags. On a hit, move the tag to most recently used. On a miss, insert it, evicting the least recently used tag only when necessary. The model is read-only, initially cold, has no prefetcher and treats requests serially. Those are experimental controls, not descriptions of all real caches.

For B=64, S=2, W=1, the trace [0, 64, 0, 128, 0, 64] produces M, M, H, M, M, H: four misses. Blocks zero and two compete for set zero. With the same S and W=2, the trace has three compulsory misses and three hits. Notice that this change also doubles capacity. To isolate associativity, compare equal total line counts by reducing the number of sets as ways increase. A “better associativity” claim that quietly adds capacity confounds two factors.

From misses to time: mark the inference

Let every access pay a lookup cost h, and each miss pay an additional penalty p. Then serial modeled cycles are N*h* + *M*p. For six accesses, four misses, h=1 and p=20, the total is 86. If the penalty is instead defined as a total miss cost, the correct expression becomes (N−M)*h* + *M*p. These two parameter conventions are not interchangeable.

The first equation omits overlap, memory-level parallelism, queueing, dirty writebacks and translation. A trace with no dependency information cannot distinguish six independent loads from a pointer chain using the same addresses. Therefore the reference outputs explicitly say “modeled serial cycles.” Modern simulation systems expose many CPU and memory models precisely because one abstraction does not answer every question. [S03, S04]

Design a sensitivity sweep

Use a sequential stream, a repeated working set and an adversarial conflict pattern. Sweep at least three capacities and two penalties. For each result, retain configuration, trace hash, warmup rule, request count and raw hit/miss counts. The reference demonstration starts cold; a warmup extension must carry cache state into the measured region without counting warmup accesses. Do not reset the cache between warmup and measurement.

A sanity bound helps catch nonsense before comparison with hardware. Misses must lie between zero and N. With a cold cache and no preloaded data, the first access to every distinct block misses. A trace fitting a fully associative cache should hit after its compulsory accesses if the working set is unchanged. A measured performance estimate must also respect relevant compute and bandwidth ceilings; the original Roofline work is a useful source for that separate bound. [S06]

Report disagreement usefully

Suppose the toy model predicts a 20% reduction in serial memory time but a real kernel improves 2%. Candidate explanations include overlap, compute domination, prefetching or a mismatch between trace and execution. Measure an independent discriminator before choosing a story. A model that misses absolute time may still rank a narrowly defined design set correctly, but that ranking needs its own validation. Preserve discrepancies rather than adjusting the plot until it looks convincing.

Practical assignment

Run a cache, scheduling or accelerator study with at least one validation case and a sensitivity sweep. Pass: raw configurations reproduce results, modeled versus measured quantities are labeled and conclusions do not exceed the tested fidelity.

Use Lab 46.3 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R38: simulation models, statistics and configuration; R48: architecture evaluation; R43: analytical bounds.

Expansion sources: S03, S04, S06. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

46.3.Q1 Why does changing from two direct-mapped sets to two two-way sets fail to isolate associativity?

46.3.Q2 Under the additional-miss-penalty convention, what is the modeled cost for N=100, M=10, h=2 and p=30?

46.3.Q3 How do verification, calibration and validation differ in an architecture study?

Lesson 46.4

Memory systems, chiplets and multi-objective design

Learning outcome: Choose an architecture across performance, power, cost and complexity.

An architecture decision is a constrained vector problem

Gaming hardware must satisfy several requirements at once: response time, throughput, energy, thermal limits, area, packaging complexity and cost. A design with the fastest isolated kernel may be unacceptable under a sustained power limit or a memory budget. Define the objective direction for every column. Milliseconds, joules per task and square millimeters are normally minimized; operations per second is maximized. Convert consistently before applying a generic comparison.

Design A dominates B if A is no worse in every objective and strictly better in at least one. Exact duplicates do not strictly dominate each other. A Pareto frontier contains designs not dominated by any other candidate. It does not identify a unique winner. Hard constraints first remove infeasible designs; preference among remaining tradeoffs requires an explicit policy. Mixing a hard thermal limit into an arbitrary weighted score can accidentally select a prohibited design.

Worked frontier

Consider synthetic candidates: A=(10 ms, 40 W, 20 area units), B=(8 ms, 50 W, 24), C=(12 ms, 45 W, 22), and D=(9 ms, 35 W, 25). C is dominated by A because A is better in all three quantities. A, B and D are non-dominated. With a 42 W power ceiling and 24-unit area ceiling, A is the only feasible candidate. Raising the area ceiling to 26 admits D, which is faster and lower-power than A but larger. A policy that minimizes latency among feasible designs then switches to D.

For steady workload duration t, energy is P*t. Here A uses 0.4 J and B also uses 0.4 J, while D uses 0.315 J. Power and energy are related but not interchangeable: a bursty high-power design can use less total energy if it finishes sufficiently sooner. The simple multiplication requires matching power to the measured interval; mixing idle-inclusive time with active-only power creates a misleading result.

Memory placement and chiplets

Splitting a system across dies can change locality, bandwidth, packaging overhead and verification obligations. Treat a cross-die access as a data movement with both fixed latency and size-dependent transfer time. An elementary lower bound is t >= L + bytes/BW for one serialized transfer, before queueing and contention. A stream of independent transfers can amortize latency differently from a dependency chain. A cache or scratchpad may reduce bytes but add area and software complexity.

Suppose a hypothetical link sustains 64 GB/s in one relevant direction and a workload transfers 8 GB per second of game time. The bandwidth fraction alone is 12.5%; it says nothing about short bursts, competing traffic or latency-critical round trips. These numbers are invented for dimensional reasoning, not specifications of a product or standard. Include metadata, retries and coherence traffic when estimating bytes. Do not assume a quoted physical signaling rate equals usable payload bandwidth.

Workload weighting can conceal regressions

A geometric mean of speedups treats ratios symmetrically but does not express a frame deadline or player-impact distribution. An arithmetic mean of milliseconds weighs long workloads more heavily. A tail percentile needs an explicit sampling unit and enough evidence in its tail. Keep per-workload values beside aggregates. A 5% aggregate gain that causes a large regression on a required adverse scene may fail the project even if the mean improves.

Architecture study is quantitative argument, not shopping by one benchmark. Historical architecture material provides methods; the student's measured limits and workload definitions determine the current study. [S05, S06]

Account for uncertainty before declaring dominance

Suppose A's measured latency is 10.0 ± 0.6 ms and B's is 9.8 ± 0.6 ms. A point-estimate frontier can label one preferable even though repeated runs overlap substantially. Report the point-estimate frontier, then show whether plausible uncertainty or changed priorities reverse the choice. Confidence intervals are not hard probability boxes, and overlapping intervals alone do not perform a paired comparison. Use a prespecified paired analysis where appropriate.

Before the lab: recompute feasibility for a 36 W ceiling and explain why a fast infeasible design should not win through weighted averaging.

Practical assignment

Construct a Pareto comparison of at least three architecture configurations. Pass: objectives and assumptions are explicit, dominated designs are identified correctly and a recommendation changes appropriately when a hard constraint changes.

Use Lab 46.4 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R48: memory and quantitative design; R49: packaging and system tradeoffs; R38: architecture alternatives.

Expansion sources: S05, S06. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

46.4.Q1 Which of A, B, C and D is dominated in the worked example, and by which design?

46.4.Q2 Why does a 42 W/24-area constraint select A, while a 42 W/26-area constraint can select D?

46.4.Q3 What information is missing when someone uses peak link bandwidth alone to predict frame latency?

Module 47

VLSI, timing and physical implementation

Prerequisites: Modules 31-34, 40 and 46. Use educational libraries and licensed or open tools only.

Module project: Carry a verified small RTL block through an educational synthesis and physical-design flow. Report constraints, timing, area and power-estimation assumptions. The result is a learning artifact, not fabrication-ready commercial signoff.

The detailed project acceptance record is in the Lab Workbook. Complete all four lessons before the integrated review.

Lesson 47.1

CMOS logic, transistor sizing and delay models

Learning outcome: Connect gate-level choices to electrical load and timing.

Connect a logic decision to an electrical load

An RTL “not” operation does not contain a delay. Physical delay arises when transistors charge or discharge capacitance through finite resistance. A CMOS inverter uses complementary pull-up and pull-down networks. Increasing device width can reduce effective on-resistance, but also increases input and diffusion capacitance. The preceding stage then sees a larger load. Optimizing one gate in isolation can make the path worse.

Use a deliberately normalized model before reading a cell library. Let a gate of relative size s have resistance R0/s, input capacitance Cin0*s and output parasitic capacitance Cp0*s. A first-order transition delay is proportional to (R0/s)(*Cload+Cp0*s). The output-parasitic term does not disappear as s grows, and the upstream penalty is absent from that single-stage equation. Constants depend on transistor behavior and waveform thresholds; do not reinterpret normalized results as nanoseconds without characterization.

Derive a tapered chain

For N inverters, define each stage's electrical effort as h_i=Cout_i/Cin_i. The product of efforts is the total load ratio H. For identical inverter types in the simplified logical-effort model, stage delay is h_i+p in normalized units. At fixed N and H, equal efforts minimize the sum: h_i=H^(1/N). This follows from the arithmetic-geometric mean inequality for positive h_i. The resulting estimate is D=N*(H^(1/N)+p).

For H=64 and p=1, one stage gives D=65, two stages give 18, three give 15, and four give approximately 15.314. The three-stage chain is best among these candidates under this model. With a fixed first-stage input capacitance of one unit, sizes 1, 4 and 16 drive a final load of 64 units. Each stage sees effort four. If the interface requires non-inverting behavior, an odd inverter count is not a valid functional substitute; parity of inversions is a hard constraint.

What the model leaves out

Rise and fall behavior need not match. Input slew affects delay; load is not necessarily a lumped capacitor; wires have distributed resistance/capacitance; device choices affect leakage and noise margin. Real libraries characterize cell arcs across input slews and loads at stated process, voltage and temperature conditions. Interpolating within characterized tables is different from extrapolating far outside them. A normalized inverter exercise teaches tradeoffs, not transistor signoff. The Cornell ASIC course and OpenSTA documentation connect the electrical model to implementation and characterized timing inputs. [S07, S08]

A library comparison experiment

Choose an educational inverter family and record the library file hash, units and corner. For several output loads and input slews, compare at least three drive strengths. Record both cell delay and input capacitance. Then build a two-stage path and repeat: the first stage must now drive the chosen second stage. The best isolated output driver may no longer be best for total delay.

A useful table has input slew, output load, cell choice, rise delay, fall delay and input capacitance. Keep maximum transition and capacitance violations visible. A cell reporting an attractive delay outside its allowed load range is not a valid winner. When a SPICE or library flow is unavailable, the normalized lab can be completed as preparation, but the external comparison remains a separately recorded outstanding requirement.

Connect back to game hardware

A command-buffer controller can spend substantial time driving high-fanout enables or long interconnect rather than performing arithmetic. Replication or buffering can reduce fanout at a cost in area, power and sometimes clock load. Pipelining changes the latency contract, so it requires more than an electrical comparison. The correct question is not “Which gate is fastest?” but “Which legal implementation meets the end-to-end contract under the specified electrical environment?”

Before the lab: calculate the four candidate chain delays, explain the load seen by the original driving stage, and identify which chain lengths preserve the required polarity.

Practical assignment

Analyze and simulate an inverter chain or compare library-cell drive strengths under several loads. Pass: assumptions are stated, input loading is included and model predictions are compared with available characterized or simulated results.

Use Lab 47.1 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R49: CMOS devices, combinational circuits and delay; R30: MOS circuit foundations.

Expansion sources: S07, S08. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

47.1.Q1 Why can increasing an output gate’s drive strength worsen the complete path?

47.1.Q2 For H=64 and p=1, what is the three-stage equal-effort delay and what are the normalized sizes?

47.1.Q3 Why is a delay reported in normalized units not a measured nanosecond delay?

Lesson 47.2

Static timing analysis, clocks and timing exceptions

Learning outcome: Check every required path under the correct timing contract.

Timing is a contract between launch and capture

A setup check asks whether data arrives early enough before a capturing clock edge. A hold check asks whether old data remains stable long enough after the relevant capture edge. They use different extremes of delay and different relationships to clock skew. Checking only the longest path misses fast-path failures.

Define positive skew k as capture-clock arrival minus launch-clock arrival. For a one-cycle same-clock path with period T, maximum clock-to-Q qmax, maximum combinational delay dmax, setup requirement s and setup uncertainty us, a simplified setup slack is T+k−us−s−qmax−dmax. For the corresponding same-edge hold check with minimum clock-to-Q qmin, minimum combinational delay dmin, hold requirement h and hold uncertainty uh, hold slack is qmin+dmin−k−h−uh. These equations use a consistent sign convention and omit common-path pessimism removal and advanced variation modeling.

Worked timing budget

Let T=2.0 ns, k=0.1 ns, qmax=0.12 ns, dmax=1.50 ns, s=0.15 ns and us=0.08 ns. Setup slack is 0.25 ns. For qmin=0.05 ns, dmin=0.08 ns, h=0.06 ns and uh=0.03 ns, hold slack is −0.06 ns. The same path can pass setup and fail hold. Tighten T to 1.6 ns: setup slack becomes −0.15 ns, while the simple same-edge hold calculation is unchanged.

Positive capture skew helps setup by moving the next capture edge later. It hurts hold by moving the same-cycle capture edge later. Increasing clock period usually helps setup; it is not a general hold repair. Adding data-path delay may repair hold but consume setup margin. A clock-tree change can affect both. Always inspect the tool's actual launch/capture edges before applying these simplified formulas to generated clocks or multi-cycle paths.

Constrain every required interface

A clock declaration alone leaves much of a block's environment unspecified. Input delays state when external data may arrive relative to a reference clock. Output delays reserve time for an external receiver. Both minimum and maximum constraints matter. Drive strength or input transition and output load describe electrical assumptions. The bundled FIFO uses a synchronous reset, so reset is an ordinary timed input in the sample constraints; it is not automatically false-pathed.

OpenSTA accepts timing libraries, netlists, constraints and parasitic information and reports checks within that model. Tool output depends on these inputs, so archive them together. [S08] A report of “no violating paths” is meaningless when intended endpoints are unconstrained or a broad exception removes the path. Require an endpoint inventory and review clock, input-to-register, register-to-register and register-to-output coverage.

Exceptions are proofs about usage

A false path declares that a timing relationship is not required by the functional contract; it does not make a difficult path disappear physically. A multi-cycle exception changes the permitted capture relationship and may require corresponding hold treatment. The exact syntax and edge effects are tool- and constraint-dependent. Never copy a multi-cycle command from another project without deriving the launch/capture schedule.

For a crossing between asynchronous domains, ordinary setup closure to the unrelated clock is not the protocol proof. A synchronizer, handshake or FIFO plus physical implementation constraints is needed. Cutting all paths between domains without identifying the synchronizer structure can hide a real multi-bit coherence problem. Clock-domain-crossing analysis and static timing are complementary.

Adversarial checks improve confidence

Intentionally tighten the clock until a known path fails. Remove an input constraint and verify that the coverage review detects the missing endpoint. Add a broad false path only in a disposable experiment and show how a superficially “green” report becomes less meaningful. Restore the legitimate constraints before any implementation comparison.

Before the lab: calculate both slacks by hand and explain why a 0.4 ns period increase cannot repair the illustrated hold failure.

Practical assignment

Constrain a small sequential block and inspect worst setup and hold paths. Pass: no unexplained unconstrained endpoints, every exception is justified and a deliberately tightened clock produces an understood violation.

Use Lab 47.2 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R39: timing analysis and flow constraints; R49: sequential timing and clocking.

Expansion sources: S08. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

47.2.Q1 Using the worked values, what are setup and hold slack at T=2.0 ns?

47.2.Q2 With positive skew defined as capture minus launch arrival, why does positive skew help setup but hurt hold?

47.2.Q3 Why is “zero violations” insufficient when assessing a timing report?

Lesson 47.3

Synthesis, placement, clock trees, routing and physical checks

Learning outcome: Understand how a logical netlist becomes a physical layout.

Preserve function while changing representation

The same hardware block appears as RTL, a generic logical netlist, a mapped cell netlist, placed cells, routed wires and a final physical database. Each representation supports different checks. A simulation pass on RTL does not establish routed timing. A clean geometric routing check does not establish FIFO ordering. Build a chain of evidence instead of treating a final layout picture as proof of everything.

The teaching block is a small synchronous ready/valid FIFO. Its protocol is deliberately conservative: input acceptance requires not-full before the edge, and output acceptance requires not-empty before the edge. At full, a simultaneous pop does not also admit a push in this implementation; there is a throughput bubble. Changing that policy is allowed only with a revised contract and tests. Fix this behavior before comparing physical results, otherwise two “FIFO implementations” may deliver different service.

What each stage contributes

Synthesis elaborates parameterized RTL, optimizes logic and maps it to available cells. Floorplanning chooses the core region, pins, macro positions and power infrastructure. Placement locates cells and exposes wire-distance costs. Clock-tree synthesis distributes the clock using buffers and routing. Routing creates interconnect that must respect the selected technology rules. Extraction estimates interconnect parasitics for timing and other analyses.

OpenROAD Flow Scripts provides an educational route through these stages and preserves reports, intermediate databases and results. The official tutorial and new-design instructions are the authority for the installed revision's commands and directories. [S09, S10] The package supplies an example configuration, not a bundled toolchain or fabricated routed layout. Pin the flow commit, platform revision, tool versions and relevant library hashes before the first comparison.

Worked interpretation: a design worsens after routing

Assume a synthetic summary shows pre-placement setup slack +0.20 ns, post-placement +0.08 ns and post-route −0.05 ns. Do not conclude that routing “broke the RTL.” The logical function may be unchanged while estimated wire delays become more realistic. Inspect the critical path: did fanout, wire length, buffering, clock skew or a changed corner account for the difference? A path can also change identity between stages, so subtracting two worst-slack values is not necessarily the delay increase of one fixed path.

Track a selected endpoint pair as well as the worst path in each stage. Compare cell delay and interconnect delay. Then change one factor, such as placement density or a high-fanout buffer strategy, and rerun under otherwise identical conditions. A speed improvement achieved by adding a pipeline stage also changes latency and reset/flow-control behavior, so it requires new functional verification and is not a pure placement experiment.

DRC, LVS and functional equivalence are different

Design-rule checking examines geometry against the available rules. Layout-versus-schematic checks compare extracted connectivity with an intended netlist. Logical equivalence compares functional representations under assumptions. Timing analysis checks path delays against constraints. Power-integrity analysis concerns the supply network. Passing one does not subsume the others, and an educational flow may not provide complete foundry-qualified decks.

Use a coverage table: performed, passed/failed, tool and revision, rule/library identity, report path and exclusions. “Not performed” is an acceptable honest entry; silently implying it was performed is not. A generated GDS file is a file format output, not authorization or evidence that a design is fabrication-ready. Review licensing before distributing any platform files; only the original teaching RTL and configuration are bundled here.

Make comparisons reproducible

A physical run is a function of more than RTL. Placement seed, thread count, optimization flags, constraints, platform files and tool versions can change the outcome. Archive logs and normalized summary tables, but keep the raw reports. Record tool failures and resource exhaustion separately from design infeasibility. A run killed by insufficient memory is not evidence that the timing target is impossible.

Before the lab: create a row for every flow stage and write one claim it can support and one claim it cannot support. Then verify the FIFO's full-and-pop behavior in simulation before synthesis.

Practical assignment

Run a small block through an educational OpenROAD-compatible flow or inspect a provided open example. Pass: capture stage reports, identify a routing/timing change and state which physical checks were and were not performed.

Use Lab 47.3 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R39: flow stages, placement, clock tree and routing; R49: physical-design methodology and EDA.

Expansion sources: S09, S10. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

47.3.Q1 Why should a FIFO’s full-and-simultaneous-pop behavior be fixed before comparing implementations?

47.3.Q2 Why can the worst setup slack change after routing without a change in RTL function?

47.3.Q3 What does producing a GDS file fail to establish?

Lesson 47.4

Power, area, process corners and design-space closure

Learning outcome: Report PPA with enough assumptions to make comparison meaningful.

Define the comparison unit

Power, performance and area are meaningful only for comparable work. A pipeline that processes twice as many items can consume more watts and fewer joules per item. A smaller block may require extra cycles, additional buffering or more host work. State the workload, throughput, latency, clock target, precision and correctness requirements before extracting a PPA table.

Keep area categories separate. Sum of standard-cell areas is not core area, die area or packaged product footprint. Core utilization is a relationship between occupied cell area and the chosen region, with details depending on what the flow counts. Comparing one run's cell area with another run's die area creates a false improvement. The physical reports and their definitions, not a spreadsheet column name alone, determine what is being compared. [S09]

Derive an energy estimate

Under a common switching convention, dynamic charging power is alphaCV^2*f, where alpha counts average charging events per clock cycle, C is effective switched capacitance, V is voltage and f is frequency. Internal cell power and leakage may be reported separately by a library-based tool. A different definition of switching activity can move a factor of two, so state the convention.

For alpha=0.2, C=10 pF, V=1 V and f=100 MHz, the estimate is 0.0002 W, or 0.2 mW. At V=0.8 V with the same alpha, C and f, it becomes 0.128 mW. That calculation does not establish that the circuit can still meet 100 MHz at the lower voltage. Slower devices can violate timing; leakage and internal power also do not necessarily scale with this simple expression.

Worked throughput comparison

Synthetic design A uses 2 mW and processes one million items per second: energy is 2 nJ/item. Design B uses 3 mW and processes two million items per second: energy is 1.5 nJ/item. B has higher power and lower energy per item. If the sustained power ceiling is 2.5 mW, B is nevertheless infeasible. If the workload arrives slowly and B can idle effectively, a whole-interval energy measurement could change the result again. Define active, idle and transition intervals explicitly.

Activity-based estimates need representative stimulus. An all-zero testbench may switch very little; random toggling can switch unrealistically much. Use at least a representative stream and an adverse high-activity case, and record the signal-activity coverage. Unannotated nets are not “zero power”; a tool may apply defaults. A credible report names those defaults and the fraction of the design they affect. [S08]

Corners and closure

Process, voltage and temperature conditions change delays and power. There is no universal rule that one named corner is worst for every check. Minimum-delay hold analysis and maximum-delay setup analysis require appropriate conditions, and temperature inversion or other effects can complicate intuition. Use the corners supplied and supported by the selected educational platform, and explicitly list missing corner coverage. Never manufacture a missing library by relabeling an existing one.

A closure loop is a controlled engineering process: identify the failing requirement, locate the responsible path or resource, propose a legal change, rerun verification, rerun implementation, and compare under the same assumptions. If a change fixes setup but breaks hold, the design is not closed. If it meets timing only under a new voltage or looser load assumption, report the changed contract rather than calling it a pure implementation win.

A bounded final claim

A suitable statement is: “Under library X, corner Y, clock Z, workload W and flow revision R, implementation B reduced reported cell area by 8% while retaining the same transaction contract; power is an activity-based estimate with 92% annotation coverage.” The numbers in that sentence are a reporting example, not package results. A poor statement is “We designed a commercially ready gaming chip.” The student's actual report should contain supported quantities and frank exclusions.

Practical assignment

Compare two verified RTL implementations using identical flow assumptions. Pass: report area, worst timing and activity-based power estimates where supported, with uncertainty and no unsupported commercial-silicon claim.

Use Lab 47.4 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R49: power, timing and implementation tradeoffs; R39: reports and analysis; R30: energy and electrical models.

Expansion sources: S08, S09. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

47.4.Q1 Compute the dynamic-power estimate for alpha=0.2, C=10 pF, V=1 V and f=100 MHz.

47.4.Q2 Why can a 3 mW design be more energy-efficient than a 2 mW design?

47.4.Q3 What must be held constant or disclosed when comparing PPA for two RTL implementations?

Module 48

Verification, reliability and hardware trust boundaries

Prerequisites: Modules 33-35, 46-47.

Module project: Create a verification dossier for the hardware capstone: requirements-to-tests traceability, assertions, coverage, clock/reset review, fault injection and a clearly bounded assurance claim.

The detailed project acceptance record is in the Lab Workbook. Complete all four lessons before the integrated review.

Lesson 48.1

Formal properties, abstraction and assume-guarantee reasoning

Learning outcome: Prove useful properties without hiding failures in assumptions.

A property needs an environment

An assertion states behavior that must hold. An assumption restricts behaviors the environment may supply. A cover goal asks whether some behavior is reachable. These are different questions. A queue can satisfy “never overflows” because the implementation is correct, or because the environment is forbidden from ever requesting a write. Only the first explanation supports a useful claim about boundary handling.

Begin with a transition system. Let queue state be an ordered sequence q of at most D items. Requests are push(value), pop and idle, with simultaneous push/pop described explicitly. Accepted transactions, rather than raw requests, update occupancy. For the conservative reference FIFO, push is accepted only when the pre-edge occupancy is less than D; pop is accepted only when it is greater than zero. Thus occupancy_next = occupancy + accepted_push − accepted_pop. At full with both requests, only pop is accepted. At empty with both requests, only push is accepted.

Derive invariants and distinguish their strength

A range invariant is 0 <= occupancy <= D. It does not prove data ordering: an implementation can keep the correct count while returning the newest rather than oldest item. Add conservation of accepted minus consumed transactions, equality of flags to boundary conditions, and a data-order property or a reference scoreboard. A hardware assertion on count and a software scoreboard on data are complementary evidence, not interchangeable claims.

Induction has a base case and a step case. Reset establishes an empty legal state. Assuming the invariant and legal transition rules, show that each combination of accepted operations preserves the range. For a finite abstract model, breadth-first exploration to a fixed point can enumerate all reachable states. That proves the checked property for that finite abstraction, provided the transition relation is correct; it does not prove every parameterized RTL implementation or analog circuit.

Bounded checking is not unbounded proof

A bounded model checker searches for a violation up to a chosen depth. A successful depth-20 check says no counterexample was found within those modeled steps, not that a failure at step 21 is impossible. Induction or another complete proof procedure may establish an unbounded result under its assumptions. A timeout is unknown. A failed property provides a counterexample; it might indicate a design bug, a bad specification or an unrealistic environment.

The SymbiYosys documentation distinguishes assertions, assumptions, covers and verification modes. The optional scripts in this bundle follow those concepts, but their execution status is recorded separately from Python tests. [S11, S12] The supplied RTL properties focus on occupancy and accounting; the simulation scoreboard checks ordering on tested traces. Do not combine them into a claim of unbounded proof of all FIFO behavior.

Worked vacuity test

Suppose a property says “when push and full, occupancy does not increase.” An assumption that push implies not-full removes every triggering full-write case. The property can pass vacuously. Add a cover goal for full && push and inspect whether it is reachable. Next, mutate the implementation so that full writes incorrectly increment occupancy. The verification should fail under the intended environment. If it still passes, investigate constraints and property activation before celebrating.

A safety property says a bad state never occurs. A liveness property says something eventually occurs. A consumer that never becomes ready can prevent eventual dequeue without violating data integrity. To prove progress, state a fairness or bounded-service assumption, then ask whether the actual system supplies it. An assumption that the consumer is ready at least once every K cycles is much stronger and more testable than an unexplained “eventually.”

Write a verification dossier

Every requirement gets an ID, property or test, environment assumptions, coverage evidence, result category and report path. Ask a reviewer to inspect assumptions before results. List properties outside the abstraction: metastability, timing, power loss, unmodeled input encodings and external memory ownership. A precise small proof is valuable; a broad claim with hidden assumptions is not.

Before the lab: write one count property and one ordering property, then explain a buggy implementation that passes the first and fails the second.

Practical assignment

Prove a queue, arbiter or transaction property and inject a violation. Pass: assumptions are reviewed independently, contention is covered and bounded, proven, failed and unknown results are reported separately.

Use Lab 48.1 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R50: safety, liveness, induction, cover and assumptions; R49: verification methodology.

Expansion sources: S11, S12. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

48.1.Q1 Why does occupancy staying in range fail to prove FIFO ordering?

48.1.Q2 What is the difference between a depth-20 bounded pass, an unbounded proof under assumptions, and a timeout?

48.1.Q3 How can a cover goal and a deliberately injected bug expose a vacuous property?

Lesson 48.2

Clock/reset crossings and asynchronous FIFO verification

Learning outcome: Check digital protocol correctness while acknowledging analog risk.

Digital agreement across unrelated clocks

A fast source can create a pulse that a slow destination never samples. Sampling each bit of a changing multi-bit value through independent synchronizers can produce an incoherent combination. A clock-domain crossing therefore needs both an event-transfer protocol and physical synchronization design. Passing ordinary RTL simulation only establishes behavior in its digital abstraction.

The reference lab uses a one-entry request/acknowledgment toggle handshake rather than pretending to supply a production asynchronous FIFO. Source-domain readiness is req == synchronized_ack. On acceptance, the source stores a payload and toggles req. It then holds the payload stable until acknowledgment returns. The destination synchronizes req, recognizes a difference from its acknowledged toggle, consumes the payload only when ready, and updates ack. Ack crosses back through a separate synchronizer. No second source transaction may overwrite the mailbox while the first remains outstanding.

Define simultaneous-edge semantics

The digital model has two-stage synchronizers in both directions. At a source edge, stage one samples the old acknowledgment and stage two samples the old stage one. The source uses the old stage two for its readiness decision. The destination performs the analogous operation for request. When edges coincide, all next-state calculations use one pre-edge snapshot. Updating one domain first and allowing the other to observe those new values in the same model step accidentally removes a cycle of crossing latency.

Take source edges every two ticks and destination edges every three ticks. After source acceptance at tick zero, request reaches destination synchronizer stages on subsequent destination edges, and consumption occurs only after the modeled synchronized value is visible to the destination logic. The exact tick depends on initial phases and the stated evaluation order. Predict the trace rather than asserting that “two flops means exactly two clocks” for every observation boundary.

From a mailbox to an asynchronous FIFO

A FIFO allows multiple outstanding entries. A common structure maintains binary pointers locally for addressing and Gray-coded pointers for crossing. Gray encoding g=b XOR (b>>1) changes one bit between adjacent binary counter values. The pointers include a wrap bit so that equal address portions can be distinguished across full and empty conditions. Empty compares the read pointer with a synchronized write pointer; full comparison must account for the appropriate wrapped pointer relationship. Derive that comparison for your chosen width rather than copying an unexplained bit inversion.

Gray coding does not make physical skew irrelevant. Each source increment changes one bit, but badly constrained routing can allow changes from successive increments to arrive out of order. Synchronizer placement, delay/skew constraints and device-specific analysis still matter. Device documentation treats multi-bit crossings and mean-time-between-failure analysis as distinct implementation concerns. [S13, S14]

Reset is a protocol event

If one side resets its toggle while the other retains an old value, an old transaction can look new or a real transaction can disappear. The bundled model requires coordinated reset of both domains and explicitly flushes the current epoch. It does not promise delivery across reset. Tag records by epoch, count in-flight dropped items, and restart only after the documented reset/release procedure. A unilateral-reset test is rejected as outside this reference contract; supporting it requires an additional resynchronization protocol.

An asynchronous reset's deassertion has recovery/removal and synchronization concerns that a discrete Python model cannot establish. In a real implementation, apply device/library guidance and verify reset-domain crossings. Do not treat a software call that sets state to zero as proof of safe physical reset release.

Separate safety from progress

No loss or duplication within an epoch is a safety/accounting claim. Eventual delivery requires clocks to keep running and the receiver eventually to accept data. Backpressure can legally keep a mailbox occupied forever under a stalled consumer. A stopped destination clock should not corrupt the held payload; it can legitimately prevent completion.

Before the lab: trace one transfer, one blocked receiver and one coordinated reset while a payload is outstanding. Name what is delivered, retained or intentionally discarded in each case.

Practical assignment

Model an asynchronous FIFO or event handshake with independent clocks and reset sequences. Pass: no loss/duplication under the stated protocol, full/empty behavior is checked and physical synchronizer assumptions are listed separately.

Use Lab 48.2 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R50: multiclock modeling and verification; R49: sequential circuits and synchronization; revisit Module 32 timing boundaries.

Expansion sources: S13, S14. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

48.2.Q1 Why must the source hold mailbox data stable until the acknowledgment returns?

48.2.Q2 Why does Gray-coding a FIFO pointer not eliminate the need for physical CDC constraints?

48.2.Q3 What guarantee does the reference handshake make across a coordinated reset, and why is unilateral reset rejected?

Lesson 48.3

Fault models, ECC, testability and recovery

Learning outcome: Design for detectable failures rather than assuming perfect hardware.

Start with an explicit fault model

A code's guarantee is always conditional on what can go wrong. A transient single-bit flip in a stored word differs from a stuck-at address line, a burst corrupting several bits, a logic bug or a stale pointer. Error-correcting codes can protect a represented word without protecting the controller that selects it. State whether faults affect data, parity, addresses, control or multiple components.

An even-parity bit makes the total number of ones even. Any odd number of bit flips changes parity; an even number can escape detection. Parity detects but does not locate a single error. Recovery might retry a read from a trusted source, discard a corrupted frame tile, restart a bounded computation or quarantine a resource. Correction is not required for every application, but an explicit response is.

Construct an extended Hamming code

The teaching code stores four data bits in positions 3, 5, 6 and 7 of a seven-bit Hamming word. Positions 1, 2 and 4 hold parity checks over indices whose corresponding binary bit is one. Position 8 is an overall parity bit over the other seven. Bit position one is the least-significant bit of the integer representation, a convention you must preserve in tests.

For each parity position p, XOR the bits at positions i where i & p is nonzero. The three resulting check bits form a syndrome. A single flip in positions 1–7 yields its position as the syndrome. The overall parity mismatch distinguishes an odd total number of flips from an even total number. For this extended (8,4) code, the supported guarantee is correction of any single-bit error and detection of any double-bit error in the codeword.

Derive the decoder table

Syndrome zero with even overall parity means no detected error under the supported model. Nonzero syndrome with odd overall parity identifies a correctable single error among positions 1–7. Zero syndrome with odd overall parity identifies the overall parity bit as the single-error candidate. Nonzero syndrome with even overall parity means a detected double-bit error and must not return trusted corrected payload.

The phrase “no detected error” is important. Higher-weight errors can transform one valid codeword into another. A triple-bit error may resemble a single-bit error, causing a conventional decoder to miscorrect. The decoder cannot infer an arbitrary fault count from eight received bits. The returned status is a classification under the stated fault model, not an oracle about what physically occurred.

Worked example and exhaustive testing

For payload 0b1010, assign data bits least-significant first to positions 3,5,6,7: 0,1,0,1. Parity positions become p1=0, p2=1 and p4=0; overall parity is one. The encoded integer is 0b11010010, or 0xD2. Flip position five: the syndrome is five and overall parity is odd; correction returns payload ten. Flip any two distinct positions: the decoder must report an uncorrectable double error and withhold trusted payload.

There are 16 payloads, 8 single-bit locations and 28 unordered double-bit pairs. Exhaustively test 128 single-error cases and 448 double-error cases, as well as all 16 clean words. Then enumerate triples to find and retain a miscorrection counterexample. That counterexample narrows the claim; it does not contradict the supported single/double guarantee.

Detection must lead to action

The Linux EDAC documentation distinguishes correctable and uncorrectable reporting and provides real operational context for counters and event handling. [S15] The teaching model is not a DIMM controller, and its counters are not predicted field failure rates. A recovery design should say which data become untrusted, whether work is idempotent, how repeated faults escalate, and when a device or region is removed from service.

For a game asset cache, reloading a verified source asset may be appropriate. For persistent player state, silently continuing with an untrusted correction can be unacceptable. Fault injection belongs in a disposable software model or authorized lab, not in a live system holding important data.

Practical assignment

Implement a parity and a small single-error-correcting code model, then inject faults. Pass: enumerate the supported correction/detection claims, test multi-bit counterexamples and define a recovery response.

Use Lab 48.3 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R49: testing, DFT and reliability context; R51: error detection/correction reporting concepts; R06: coding and combinatorial reasoning extension.

Expansion sources: S15. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

48.3.Q1 What do syndrome nonzero and even overall parity mean under the SECDED fault model?

48.3.Q2 How many single-bit and double-bit cases must be tested over all sixteen four-bit payloads?

48.3.Q3 Why can a triple-bit error be miscorrected despite a correct SECDED implementation?

Lesson 48.4

Hardware security, thermal limits and assurance review

Learning outcome: Include isolation and failure containment in the design contract.

Security and reliability share boundaries, not guarantees

A game accelerator or engine plugin can receive malformed commands even when the normal application is well behaved. Treat descriptors, lengths, ownership identifiers and reset requests as untrusted inputs at a boundary. Fault containment asks what state a failed component can damage. A threat model asks who can influence that component and what capability they possess. Neither is satisfied by a fast nominal benchmark.

The local teaching controller accepts descriptors containing offset, byte length, owner and generation. It has a fixed buffer size and does not perform real DMA. Validation checks type, nonnegative offset, positive length, alignment, ownership, generation, resource limits and region bounds before any modeled write. A safe fixed-width bounds idiom is offset <= capacity and length <= capacity−offset after validating nonnegative inputs. This avoids overflow in offset+length on bounded integer machines.

Worked descriptor review

Let capacity=4096 bytes, alignment=16 and active owner=7. A descriptor at offset 4080 with length 16 fits exactly. Length 32 does not. Offset −16 must be rejected before conversion to unsigned arithmetic. Owner 8 cannot gain access merely because its numeric range fits. A stale generation from before reset must not be accepted after reset. Length zero needs a defined policy; this model rejects it to avoid ambiguous no-op accounting.

Validation alone does not stop time-of-check/time-of-use races if an untrusted producer can modify the descriptor after validation. Copy a validated descriptor into private state or use a protocol that freezes ownership. A real IOMMU or device protection mechanism needs correctly configured domains and mappings. CPU virtual addresses, physical addresses and DMA addresses are not interchangeable; the Linux DMA guide explains that distinction. [S16] The Python bounds checker proves nothing about a real machine's IOMMU configuration.

Reset under load

A reset may leave a transaction partly completed. Define whether reset drains, cancels or quarantines outstanding work. Tagging completions with an epoch can reject stale completions from an earlier generation, but does not undo bytes already written before reset. The teaching controller clears pending work and increments its generation. It models no partial external writes. A physical design with partial writes needs an additional consistency/recovery protocol.

Log rejected operations without allowing logs themselves to exhaust memory. Bound descriptor count, retry count and payload size. Test denied access as a normal expected result rather than an assertion crash. Assertions are useful for internal impossibilities; hostile inputs require explicit error handling. Limit all exercises to the local simulation and authorized artifacts.

Thermal limits are part of the operating envelope

A thermal model must state ambient temperature, cooling assumptions, power distribution and workload duration. A simple steady estimate Tj≈Ta+P*theta uses a thermal resistance theta; a transient model adds thermal capacitance and time constants. Board and package conditions determine whether that approximation is useful. Do not import a datasheet thermal resistance into an unrelated enclosure without qualification.

For an invented example with ambient 30 C, power 8 W and effective resistance 5 C/W, the steady estimate is 70 C. This is arithmetic, not a safe operating limit for any device. Trip thresholds and cooling actions are platform-specific; Linux's thermal documentation shows how zones, trip points and cooling devices are represented in software. [S17] Never bypass thermal shutdown, cooling, voltage protections or firmware limits to complete a lab.

Produce an assurance claim with exclusions

A useful claim is: “Within the modeled descriptor format and memory region, denied owners, stale generations and out-of-range transfers produce no modeled write; coordinated reset clears pending work.” A complete review then lists what remains untested: real DMA isolation, speculative side channels, analog fault behavior, thermal calibration, external memory corruption and power-failure atomicity.

Before the lab: classify a range-valid but wrong-owner descriptor and a correctly owned but stale-generation descriptor. Explain why both must fail before memory effects.

Practical assignment

Write a threat-and-failure review for the accelerator and test malformed descriptors, denied access and reset under load in simulation. Pass: isolation limits, protection assumptions and unresolved risks are explicit; no unauthorized target testing occurs.

Use Lab 48.4 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R51: device access, DMA and isolation; R49: implementation and verification; R50: enforceable protocol properties.

Expansion sources: S16, S17. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

48.4.Q1 Why is length <= capacity−offset preferable to blindly checking offset+length <= capacity on a fixed-width machine?

48.4.Q2 What does an epoch/generation check prevent after reset, and what does it not undo?

48.4.Q3 Why does a passing Python descriptor test not establish real DMA isolation or safe thermal operation?

Module 49

Master's thesis: proposal, implementation and evaluation

Prerequisites: Gate I and relevant advanced-track modules; independent technical mentorship strongly recommended.

Module project: Prepare and execute a thesis-scale project in one primary specialization. Budget 400-800 focused hours as a planning range, with scope adjusted through review. The deliverable is a defensible contribution and reproducible artifact, not a promise of publication.

The detailed project acceptance record is in the Lab Workbook. Complete all four lessons before the integrated review.

Lesson 49.1

Question selection, literature map and thesis scope

Learning outcome: Turn a broad interest into a falsifiable, feasible research question.

A thesis is an answerable question, not an oversized product

“Build a better game engine” does not identify a research claim. A useful question names a mechanism, workload, comparator, outcome and boundary. For example: “For bursty visibility-command traces, can a bounded admission policy reduce the 95th-percentile queueing delay relative to FIFO while preserving ordering constraints and a fixed memory budget?” This is a possible study question, not an established result or promise of novelty.

One primary thesis is sufficient. It may be architectural, graphics-focused, simulation-focused, compiler-focused or verification-focused, provided the selected track's prerequisites and evaluation obligations are met. The architecture, physical-design and assurance exercises can feed one project rather than requiring a new engine and a new GPU. The original program allocates approximately 400–800 focused thesis hours inside Volume 10's total estimate; those hours are not an additional charge on top of the volume.

Make a falsifiable contribution statement

Write a provisional hypothesis and a refutation condition. “Policy B is faster” is underspecified. “At equal accepted work and buffer capacity, B reduces the prespecified per-run p95 latency by at least 5% on the held-out bursty workload set, while no required scene regresses beyond the declared limit” identifies a decision threshold. The threshold is a design choice to review, not a statistical fact. A null result can be useful when the baseline is sound and the experiment genuinely tests the mechanism.

Separate mechanism novelty from evaluation novelty. Reimplementing a known method is a reproduction contribution; testing its behavior under a new carefully justified workload can be an evaluation contribution. A small extension with ablation evidence can be a method contribution. “No one has done this” is a literature claim requiring a serious search, not something inferred from a few search results.

Build a literature map of 8–15 relevant sources

For each source, record the question, method, assumptions, workload, comparator, metric, evidence type, limitations and relationship to your proposal. Read the actual method and evaluation, not just the abstract. Include the original paper behind a technique, relevant official implementation documentation and at least one source that challenges the proposal's assumptions. Follow citations backward for foundations and forward for later evaluations; record the search date because the field changes.

The inherited program references include original Roofline, ReSTIR and XPBD work as possible starting points, not requirements to combine all three. A source map can reveal that the apparent “new” algorithm already exists, or that the meaningful contribution is making a comparison reproducible. That discovery improves scope. The architecture and ASIC teaching sources help define technical background, but do not substitute for topic-specific research reading. [S05, S06, S07]

Worked narrowing exercise

Start with “hardware for better RTS performance.” Narrow the application to command dispatch for a fixed engine subsystem. Identify an observable problem: burst arrivals produce queueing delay. Choose one intervention: admission or scheduling policy. Fix boundaries: same capacity, same dependency constraints, no change in accepted work, no custom silicon required. Select a baseline and a correctness oracle. Then state the primary outcome and adverse cases: dependent commands, long tasks, saturating bursts and starvation risk.

This narrowed question is feasible in a simulator, but its conclusion concerns that modeled system unless validated against engine traces or a maintained implementation. The proposal must say which evidence tier it targets. A simulator-only study can be rigorous; pretending it measured a shipping game is not.

Scope by elimination and risk

Write an explicit exclusions paragraph: no complete GPU design, no production driver, no foundry tapeout, no commercial performance guarantee, no unauthorized proprietary traces. List compute budget, available data, tool access and reviewer availability. Choose the smallest baseline that can falsify the idea before the major implementation. If necessary data or tools are unavailable, change the question through review rather than quietly replacing evidence with invented numbers.

Before the lab: write one sentence each for the question, mechanism, baseline, metric, refutation condition and excluded work. A reviewer should be able to repeat the intended experiment without guessing the missing nouns.

Practical assignment

Write a 3-5 page proposal with a research question, 8-15 relevant sources, baseline plan, constraints and exclusions. Pass: a reviewer can explain what observation would weaken or refute the main hypothesis.

Use Lab 49.1 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

Use Module 41's paper method and the chosen track's original papers; R44, R48, R49 or R55 are starting anchors, not a substitute for the literature search.

Expansion sources: S05, S06, S07. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

49.1.Q1 What five elements make a broad “better hardware” topic into a testable thesis question?

49.1.Q2 How can a reproduction or negative result still be a defensible contribution?

49.1.Q3 Why should the literature map include a source that challenges the proposal’s assumptions?

Lesson 49.2

Proposal review, baselines and risk-reduction milestones

Learning outcome: Make failure cheap before the expensive implementation phase.

Make uncertainty cheaper before making implementation larger

A baseline is the current best supported comparator for the question, not the slowest code you can find. First make it run correctly. Then establish that the primary metric can be measured and that its variance, runtime and data requirements fit the project budget. A week spent detecting an invalid baseline can save months of meaningless implementation.

Use milestones that produce evidence rather than activity descriptions. “Read more papers” has no stopping condition. “Map ten relevant sources and identify which assumptions the baseline shares” does. “Implement accelerator” is too large. “Execute one end-to-end workload with a correctness oracle and raw timing export” is a reviewable risk-reduction milestone. Each milestone has a decision: continue, narrow, change method or stop this question.

Freeze the evaluation contract

Record the primary metric, its unit, sampling unit, aggregation rule, inclusion/exclusion rules, stopping rule, run count rationale, warmup behavior, seeds, required workloads and adverse cases. Define correctness and feasibility constraints separately from performance. Decide whether failed runs count as failures, timeouts or missing data; do not silently delete them from the final comparison.

Paired measurements compare matched conditions, such as the same scene and seed under baseline and candidate. The meaningful independent unit might be a run, machine session or scene, not each frame. NIST's discussion of paired observations provides the basic matching idea. [S18] If thousands of frames come from the same evolving simulation, treating each as independent can greatly overstate precision. Plan the hierarchy before seeing a favorable result.

Worked feasibility checkpoint

Suppose an initial synthetic plan calls for 20 scenes, three configurations, ten runs each and five minutes per run. Runtime alone is 3000 minutes, or 50 hours, before compilation, analysis and failures. If the simulator instead needs 30 minutes per run, the same plan requires 300 hours. A feasibility pilot on two scenes should estimate runtime and measurement behavior before committing the full matrix.

A run that reports a speedup is not enough. Verify equal work, deterministic correctness where required, and a valid timing region. For CPU/GPU comparisons, asynchronous work can make host submission time differ from completion time. A baseline that never waits for the device can look artificially fast or slow depending on the

comparison. Define synchronization at the measurement boundary rather than inserting arbitrary waits until the result seems plausible.

Build a risk register with triggers

Each risk needs probability/impact as qualitative planning judgments, an observable trigger, a mitigation, an owner and a review date. Example: “Baseline cannot reproduce the claimed output” has the trigger “two independent reruns fail the documented tolerance.” Mitigation might be contacting maintainers, narrowing to a verified reference or changing the question. “Simulator too slow” can trigger reducing the workload set through a reviewed sampling plan, not choosing only scenes where the candidate wins.

Separate a contingency from an undisclosed methodological switch. If measurement hardware is unavailable and the study moves to analytical modeling, its evidence tier changes. Preserve the original plan, document the amendment before running the substitute study, and narrow the conclusions. The absence of a desired result is not a reason to rewrite the primary metric after inspection.

Define the baseline acceptance review

The baseline review includes a runnable command, environment lock, data provenance, correctness check, raw output, first summary and known discrepancies. The reviewer should run at least one case independently. The candidate implementation begins only after the baseline is sufficiently trustworthy for the proposed question, or after an explicit decision to study the baseline discrepancy itself.

Before the lab: estimate the complete run matrix in wall-clock hours, identify the highest-cost uncertainty, and design the smallest pilot that can resolve it. Keep the thesis-hour range as a planning aid, not a quota to fill.

Practical assignment

Deliver a running baseline, risk register and milestone plan. Pass: the primary metric can be measured, a small feasibility experiment exists and each major risk has a mitigation or explicit acceptance decision.

Use Lab 49.2 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R49: project milestone and reporting structure; Module 41: experimental design; chosen method's primary documentation.

Expansion sources: S18. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

49.2.Q1 Why is a working baseline with a correctness oracle more valuable than a large unvalidated prototype at the first milestone?

49.2.Q2 For 20 scenes, 3 configurations, 10 runs and 5 minutes per run, what is the execution budget?

49.2.Q3 What distinguishes a legitimate protocol amendment from changing the question to fit results?

Lesson 49.3

Implementation, ablations and controlled evaluation

Learning outcome: Execute the study without changing the question to fit a favorable result.

Execute the approved question

The implementation phase produces both a method and evidence about that method. Preserve the approved protocol in version control. Each experiment records code identity, configuration, input identity, seed, environment, start/stop conditions and status. Store raw observations before analysis. Derived tables should be regenerated from raw data by scripts, not edited manually to match a narrative.

Correctness is a gate before performance comparison. A faster method that drops work, changes precision outside the contract or skips difficult scenes does not satisfy the same task. Compare completed work and output quality, not just elapsed time. Maintain a failure ledger with crashes, timeouts, invalid outputs and unsupported configurations. Those entries are part of the result.

Ablation and sensitivity answer different questions

An ablation removes or disables a proposed component while preserving a fair comparison. It asks what that component contributes. A sensitivity study varies a parameter or assumption, such as queue capacity, miss penalty or workload burstiness. It asks whether the conclusion is robust. A baseline-versus-candidate comparison alone cannot establish which of several simultaneous changes caused a benefit.

For a two-component method, run baseline, A only, B only and A+B where feasible. Suppose synthetic costs are 100, 90, 95 and 70 units. A reduces cost by 10, B by 5 and the combined version by 30, suggesting interaction under that workload. Do not simply assign the extra 15 to a universal synergy; repeat across cases and verify that the configurations perform identical work.

Paired analysis with honest units

For each matched independent unit i, let b_i and c_i be positive baseline and candidate times. The speedup ratio is r_i=b_i/c_i, so values above one favor the candidate. The geometric mean is exp(mean(log(r_i))). Equal ratios 1.25 across units give geometric mean 1.25: 25% more throughput under equal work, but 20% less time. “25% faster” is ambiguous unless the metric is named.

The bundled analysis includes a deterministic paired bootstrap: resample whole pairs with replacement, calculate the aggregate for each resample and take the declared percentile bounds. It is a teaching implementation, not a guarantee of statistical coverage for every dependence structure. Resampling individual frames when the independent units are sessions does not fix pseudoreplication. For nested data, aggregate at the prespecified independent unit or use an appropriately reviewed hierarchical/block method. The basic paired-observation principle is supported by NIST's treatment; the package's bootstrap procedure is an original worked implementation. [S18]

Worked difference between confidence and proof

Consider four synthetic pairs: baseline times [10,12,8,20] ms and candidate [8,9,7,18] ms. Their ratios are 1.25, 1.3333, 1.1429 and 1.1111. The geometric mean is about 1.206. A bootstrap interval from only four units can be unstable and cannot justify a broad population claim. Use this example to verify arithmetic, not to infer a real device improvement. The example datasets in the bundle are conspicuously labeled synthetic.

A confidence interval quantifies uncertainty under the analysis assumptions; it is not a probability that a particular engineering hypothesis is true. An interval excluding one does not establish correctness, novelty or practical importance. Keep effect magnitude, adverse-case behavior and the prespecified decision threshold beside the interval.

Freeze, rerun and explain failures

Reserve a held-out workload set for the final evaluation. Tuning on it repeatedly turns it into training data. Record protocol deviations with dates and reasons, and separate confirmatory results from exploratory analysis. A final clean run should regenerate the central table from a tagged artifact. Reproduce at least one known failure case as deliberately as a successful one.

Before the lab: explain which component the ablation removes, which assumption the sensitivity sweep changes, and which independent units your uncertainty analysis resamples.

Practical assignment

Run the approved evaluation with baselines, at least one ablation and one sensitivity study. Pass: scripts regenerate results, failed cases remain visible and claims use the preregistered primary metric or clearly disclose deviations.

Use Lab 49.3 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R09 and Module 41: uncertainty; chosen track's original methods and implementation references; handbook experiment manifest.

Expansion sources: S18. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

49.3.Q1 How does an ablation differ from a sensitivity study?

49.3.Q2 If baseline time is 10 ms and candidate time is 8 ms, what are speedup and time reduction?

49.3.Q3 Why is resampling thousands of frames from one run not equivalent to collecting thousands of independent runs?

Lesson 49.4

Thesis writing, reproducibility and contribution boundaries

Learning outcome: Write a technical argument another engineer can audit.

Write an argument that exposes its own evidence

A thesis report is a structured claim, not a diary of everything attempted. The introduction names the question and the contribution boundary. Background defines the necessary concepts. Related work explains what is inherited and what remains uncertain. Method provides enough detail to implement the approach. Evaluation tests the hypothesis against baselines and alternatives. Limitations describe where the evidence does not support extension. The conclusion answers the question at the demonstrated scope.

The original program suggests roughly 30–60 pages plus technical appendices as a planning guide. Evidence quality matters more than page count. Do not pad the report with copied tool manuals or repeated plots. Put exact commands, long proofs, raw tables and implementation details in appendices or the artifact, referenced from the main argument.

Create a claim-to-evidence ledger

Give each major claim an ID. For C1, store the exact sentence, evidence tier, required assumption, raw input path, script/command, result path, correctness check and limitation. Ask a reviewer to trace five major claims. If a sentence has no reproducible evidence path, rewrite it as a hypothesis or remove it. A bibliography citation supports prior work; it does not substitute for evidence about your own implementation.

Example: “The candidate lowers modeled serial cache cost on the conflict trace.” The ledger should point to the trace hash, cache geometry, penalty convention, raw miss counts and generating script. It must not silently expand into “The candidate improves shipped game frame rate.” Different verbs signal different evidence: derived, modeled, simulated, measured and independently reproduced are not synonyms.

Build a portable artifact

A fresh reader needs an entry document, dependencies with versions, input provenance, a short correctness run, a central-result command, expected output, runtime/resource estimates and troubleshooting. Separate student-authored code from third-party libraries and declare assistance. Keep credentials, private traces and proprietary platform files out of public archives unless authorized. Provide a legal acquisition procedure when redistribution is not permitted.

A cryptographic file hash detects changes relative to a trusted manifest. It does not prove authorship, source authenticity or scientific correctness. Pin source revisions and, where relevant, executable environments. An environment lock saying “latest” is not a lock. Platform differences may prevent bitwise equality, so define numerical tolerances or statistical reproduction criteria before the reviewer runs the artifact.

Worked reproduction contract

For a deterministic cache table, expected hit/miss counts and a CSV hash can be exact. For a timing experiment on different hardware, expecting the same nanoseconds is usually inappropriate. Define what must match: valid outputs, workload completion, configuration identity where possible, relative trend within a prespecified tolerance, and disclosed differences. A non-matching result is not automatically the reviewer's error. Record the mismatch, inspect environment and methodology, and narrow the claim if necessary.

The bundle's checksum validator verifies file integrity and the example generation scripts reproduce teaching outputs. Those checks are not a completed thesis reproduction. The learner's thesis must supply its own artifact and central result. The same distinction applies to an included answer key: it teaches expectations but is not evidence that the learner independently completed the work.

Integrate limitations rather than hiding them

Every result section should state relevant limits near the claim. A global disclaimer at the end cannot repair an overstated figure caption. Explain model omissions, workload selection, hardware availability, baseline quality, statistical assumptions and missing physical checks. A failed ablation or a regression can identify the true mechanism more clearly than an aggregate win.

Revision is part of the result

Keep reviewer comments, responses and changed claims in a revision log. Tag the final artifact only after claims, figures and code agree. Use a final audit to remove stale plots and numbers. Professional version-control practice supports reviewable history and attribution; the Git project's guidance is a useful workflow reference. [S19]

Before the lab: choose the strongest sentence in the draft and trace it all the way to raw data and a correctness check. Then write the strongest sentence the same evidence does not justify.

Practical assignment

Produce a thesis-style report, typically 30-60 pages plus appendices as a planning guide, and a complete artifact package. Pass: a reviewer traces five major claims to evidence and reproduces the central table or experiment.

Use Lab 49.4 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R49: final project reports; chosen original papers for reporting comparisons; handbook thesis outline and evidence checklist.

Expansion sources: S19. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

49.4.Q1 What is the purpose of a claim-to-evidence ledger?

49.4.Q2 Why does a matching SHA-256 file manifest not prove that a research conclusion is correct?

49.4.Q3 How should reproduction criteria differ between a deterministic cache count and a cross-machine timing experiment?

Module 50

Thesis defense, professional practice and continued development

Prerequisites: Module 49 and all prerequisites for the selected specialization.

Module project: Gate J: defend the thesis, pass an independent artifact reproduction, complete a maintainable handoff and publish an honest portfolio where rights permit. This completes the self-study program, not an accredited degree.

The detailed project acceptance record is in the Lab Workbook. Complete all four lessons before the integrated review.

Lesson 50.1

Oral defense and independent artifact reproduction

Learning outcome: Show that the learner owns the reasoning and implementation.

The defense checks ownership of reasoning

A polished slide deck is not sufficient evidence of understanding. The learner must explain the mechanism, derive a central relation, interpret a failed case and reason about a small implementation change. The original Gate J calls for a 30–45 minute defense with a qualified reviewer or peer panel, plus independent artifact reproduction and completed revisions. This is an educational assessment, not an accredited degree examination.

Use a practical 40-minute structure: five minutes for question and scope, ten for method and derivation, ten for evidence and limitations, ten for questions including a failure case, and five for an implementation or handoff question. A separate artifact run should occur before or after the meeting; forcing a long simulator run into a short defense wastes the review window. Provide the artifact and evaluation contract to reviewers in advance.

Choose a derivation that matters

For a cache thesis, derive the address mapping and timing model, then explain a counterexample to its hardware fidelity. For a hardware implementation thesis, derive a setup/hold relation or protocol invariant and connect it to actual reports. For graphics or simulation, derive the estimator, stability condition or optimization legality central to the contribution. A memorized equation without assumptions does not meet the goal.

A reviewer can ask: “Which assumption would reverse your result?” The answer should point to a sensitivity study or explain a proposed experiment, not claim that all assumptions are harmless. Another useful question is: “What can your passing test suite fail to detect?” That probes abstraction boundaries and coverage more directly than reciting the number of tests.

Worked challenge

Suppose the thesis reports a 1.20x geometric-mean speedup, but one required adverse workload regresses by 30%. The defense should show the per-workload table, explain whether the protocol allowed that regression, and state whether the thesis's acceptance criterion passed. A favorable aggregate does not erase a violated hard requirement. The contribution may remain useful as a bounded finding even if the original deployment hypothesis is rejected.

Suppose instead a reviewer regenerates the central table and obtains a different ranking. First preserve both runs. Compare code/configuration/input identities and environment. Check nondeterminism, measurement region, warmup and analysis rules. If the mismatch is unresolved, narrow or suspend the affected claim and create a revision task. “It worked on my computer” is not a defense of reproducibility.

Independent reproduction means independent operation

The reviewer should start from the release entry point on a clean checkout or extracted archive, not from the author's shell history. The author can answer documented questions, but undocumented manual edits must become artifact corrections. Record commands, environment, elapsed time, outcomes and deviations. A second run by the author on the same prepared machine is a useful check, not independent reproduction.

Score evidence, not confidence

The Gate J rubric evaluates question/scope, correctness, method, evaluation, reproducibility, defense and handoff. A calm admission that a result is unknown can be a stronger answer than a confident unsupported claim. Critical failures such as invalid output, fabricated evidence or missing central reproduction block passing regardless of total points. A reviewer can require focused revisions rather than restarting the entire thesis.

The review should distinguish work done independently, with mentorship, using AI assistance or adapted from third-party code. Assistance does not invalidate learning, but the learner must own the submitted reasoning and attribute help. Do not stage a live coding challenge that rewards memorized syntax over the central technical contract.

Before the lab: rehearse a two-minute explanation of the contribution, a five-minute derivation and a three-minute failed-case analysis. Then answer one question without the slides.

Practical assignment

Hold a 30-45 minute technical defense with a qualified reviewer or peer panel and a separate artifact run. Pass: the central result is reproduced, major questions receive evidence-based answers and required revisions are completed.

Use Lab 50.1 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R49: design-project evaluation as a reference; handbook defense questions and rubric.

Expansion sources: . See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

50.1.Q1 Why is a second run by the author on the same prepared machine not independent artifact reproduction?

50.1.Q2 What should happen when a reviewer cannot reproduce the central ranking?

50.1.Q3 Can a favorable average result override a violated hard correctness or adverse-case requirement?

Lesson 50.2

Mature-codebase integration, review and maintainable contributions

Learning outcome: Transfer a prototype into an engineering environment with other owners.

A research prototype enters a social codebase

A maintained codebase has users, conventions, interfaces, tests and other owners. An algorithmically sound change can still be unsuitable if it breaks compatibility, obscures ownership or adds an unmaintainable dependency. The integration task is a focused contribution, not a demand that an upstream maintainer accept a thesis-sized patch.

Choose a mature open project or a maintained local fork for which you have the necessary rights. Read contribution guidance, build instructions and relevant tests before editing. Identify the narrow seam: a cache replacement policy, a simulator report parser, a bounded queue check, a benchmark harness improvement or a reproducibility fix. Keep unrelated formatting, dependency upgrades and feature changes out of the patch.

Design the patch around a contract

State the existing behavior and the proposed behavior. For a bug fix, include a regression test that fails on the base revision and passes with the patch. For a performance change, show correctness equivalence and a representative benchmark with adverse cases. For an API change, identify callers, compatibility strategy, migration path and deprecation behavior. A benchmark-only change can be valuable when it corrects misleading measurement or data loss.

Pro Git describes workflows for focused contributions and reviewable history. Use the target project's actual guidance for the selected revision rather than imposing a generic branching ritual. [S19] A local maintained fork is acceptable when upstream scope or timing does not fit, provided the handoff explains the divergence and maintenance cost.

Worked patch boundary

Suppose a simulator's CSV exporter writes nonfinite values as empty strings, making analysis silently omit failed configurations. A focused patch could reject or explicitly label those results, add a test for NaN/Infinity, document the schema and preserve the failure row. It should not also rename every module and replace the build system. The research value is stronger evidence integrity, and the integration value is a small, reviewable change with clear behavior.

Another example is a FIFO throughput optimization that allows a push at full when a pop occurs. This is not necessarily a drop-in bug fix: it changes the acceptance contract and potentially combinational paths. Update

tests, documentation and timing analysis. A reviewer may prefer the conservative behavior for predictability. Technical review is negotiation around constraints, not a referendum on whether the author is clever.

Make review responses auditable

For each comment, record the concern, response, changed commit and remaining disagreement. “Fixed” is insufficient when the issue concerns a design assumption. Explain why the test covers the failure mode or why a requested change would violate a requirement. Separate factual corrections from preference differences. Keep benchmark data that contradict the preferred approach.

Continuous-integration checks should run the meaningful tests in a clean environment. A green badge can conceal skipped tests or a configuration that never builds the changed component. Record the exact jobs and coverage relevant to the patch. Tool versions and platform assumptions matter just as they did in the thesis study.

Plan rollback and ownership

Every contribution needs an owner or a documented unowned state, a rollback strategy and a known-issues record. For data-format changes, rollback may require preserving compatibility with older readers. For a hardware-flow change, old results may no longer be directly comparable. A changelog should state those consequences plainly.

Acceptance upstream is not required for the curriculum. A rejected but technically rigorous patch can still demonstrate competence when the learner documents the review, maintains a local fork and understands why it was not accepted. Do not claim maintainership, endorsement or upstream inclusion unless it actually occurred.

Before the lab: write a patch title under one sentence, name its regression test and list three changes that deliberately remain out of scope.

Practical assignment

Integrate one tested improvement into a mature open codebase or a maintained local fork. Pass: a focused patch, regression test, design rationale and reviewer response are included, whether or not an upstream project accepts it.

Use Lab 50.2 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R05: distributed workflows and contributing; R11: interfaces and maintenance; R26: mature-engine comparison where authorized.

Expansion sources: S19. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

50.2.Q1 Why should a performance patch include correctness and adverse-case evidence?

50.2.Q2 What is the strongest basic evidence for a bug-fix regression test?

50.2.Q3 Does an upstream project have to accept the patch for this lesson to be complete?

Lesson 50.3

Portfolio, technical interviews and role-specific evidence

Learning outcome: Present concrete engineering competence without inflated claims.

Convert course work into verifiable role evidence

A portfolio should answer what problem existed, what the learner personally did, what constraints mattered, how correctness was checked, what the evidence showed and what remained unresolved. It should not be a directory of screenshots labeled “advanced.” A concise case study with a runnable artifact is stronger than a broad list of technologies without demonstrated work.

Prepare three case studies: one systems/engine engineering example, one algorithm or numerical example, and one specialization/thesis example. These may come from earlier volumes. Reuse real artifacts rather than inventing three more capstones. For each, distinguish individual work from team contributions, source adaptation, mentorship and AI assistance. Do not imply accredited degree completion; describe the self-study curriculum and actual technical artifacts.

Match evidence to the role

Engine-systems evidence might include ownership/lifetime reasoning, concurrency correctness, profiling and a tested integration. Architecture evidence might include model validation, controlled workload comparisons and explicit performance/power assumptions. Hardware-design evidence might include RTL protocol tests, timing constraints, physical-stage reports and coverage limits. Research-engineering evidence might include a literature map, baseline reproduction, ablations and an independently reproduced central result.

The CS2023 knowledge-area directory offers a breadth checklist, not a hiring guarantee or certification. [S20] A role-specific case study should still acknowledge missing breadth: an excellent cache model does not establish analog design competence, and a polished graphics demo does not establish kernel-driver safety.

Worked claim rewrite

Weak claim: “Designed a next-generation gaming processor that is 30% faster.” Stronger evidence-matched claim: “Implemented and validated a read-only LRU cache model; on three disclosed synthetic traces, compared equal-capacity geometries and regenerated the miss-count table from versioned inputs. Results are analytical-model evidence, not silicon benchmarks.” A later project with real physical or engine evidence can make a stronger claim, but only to the demonstrated scope.

For a thesis, a useful case-study structure is context, hypothesis, contribution, correctness method, main result, failed case, limitations and reproduction command. A negative result belongs in the portfolio when it demonstrates good engineering judgment. Employers and collaborators need to know whether the author can identify when an attractive approach should not ship.

Three mock interviews

The systems interview asks the learner to diagnose a concurrency or memory-lifetime bug and design a measurement plan. The algorithm interview asks for a correct solution, complexity analysis, boundary tests and an alternative tradeoff. The specialization interview asks for a central derivation, model assumptions and a counterexample. Each interview includes a brief code or pseudocode exercise and a written reflection afterward.

Example systems prompt: a producer fills a shared command buffer and sets a relaxed atomic flag; the consumer reads the non-atomic payload after seeing the flag. Explain the source-language problem and repair the publication contract. Example algorithm prompt: compute a Pareto frontier, define strict dominance, analyze the quadratic baseline and test duplicates. Example specialization prompt: derive setup and hold slack under a stated skew convention, then explain why slowing the clock may not repair hold.

Evaluate thinking with observable criteria

Score problem clarification, contract, correctness, reasoning, tests and communication. Record hints used. Reattempt a related unseen problem after remediation rather than memorizing the original answer. A pass does not require perfect recall of an API, but it does require recognizing when an assumption is unsupported and finding a valid way to resolve it.

Protect private data and respect source rights when publishing artifacts. Redact secrets and proprietary traces; replace them with an authorized reproducible example without implying that the substitute is the original measured dataset. Keep the private evidence link available to authorized reviewers when appropriate.

Before the lab: inspect every numerical claim in one case study and point to the exact artifact supporting it. Rewrite any claim whose scope exceeds that evidence.

Practical assignment

Prepare three portfolio case studies and complete one systems, one algorithm and one specialization mock interview. Pass: every claimed result is supported by an artifact and assistance or third-party contributions are clearly attributed.

Use Lab 50.3 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R01-R02: breadth review; R11: engineering explanation and contracts; handbook role map and portfolio template.

Expansion sources: S20. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

50.3.Q1 What makes a portfolio case study more credible than a list of tools?

50.3.Q2 How should the learner describe completing this program?

50.3.Q3 Why can a well-explained negative result belong in a technical portfolio?

Lesson 50.4

Long-term learning, maintenance and final handoff

Learning outcome: Keep the knowledge and artifacts useful after the course ends.

Make the artifact survive its author's machine

A completed thesis is not yet a maintainable project. Another person needs to locate the release, understand its purpose, build it, run tests, reproduce the central result, diagnose common failures and know who owns unresolved work. The final handoff turns personal knowledge into project knowledge.

Create an entry document with the supported environment, minimum required tools, short verification command, full study command, expected results and approximate resource needs. Distinguish an exact reproducibility environment from other environments that may work but were not tested. Keep package-local scripts portable by resolving paths relative to their files or an explicit project root, not a personal home directory.

Archive evidence and provenance

The final archive contains the thesis, appendices, reviewed proposal, literature map, source, configurations, raw data, analysis scripts, tests, result tables, reviewer reproduction log, revisions, rights/assistance disclosure and maintenance plan. Hash the inventory. Store a separate trusted copy of the manifest; a modified artifact and modified manifest can otherwise agree with each other. A checksum is an integrity tool, not a substitute for access controls or source authenticity.

Use a release tag and a changelog. Record what future updates will invalidate: a compiler change may alter performance, a model correction may change results, a dependency change may break numerical equivalence. Preserve the historical release even after fixing bugs, with a clear correction notice when a published claim changes. Git's workflow guidance supports traceable contributions, but the project must define its own maintenance policy. [S19]

Write the maintainer runbook

The runbook includes architecture overview, data flow, public contracts, common commands, test organization, known failure modes, troubleshooting and rollback. Each known issue has severity, reproduction, impact, workaround and ownership. "No known issues" should mean the issue review found none within the stated scope, not that limitations were omitted.

For hardware artifacts, include clock/reset assumptions, tool/platform locks, constraint rationales and physical-check coverage. For simulator artifacts, include the model boundary and calibration/validation evidence. For engine artifacts, include performance measurement boundaries, data/asset requirements and compatibility assumptions. Different maintainers need different first questions answered.

Worked handoff test

Give a reviewer only the release archive and entry document. Ask them to locate the central experiment, run the smoke test and explain one key invariant. Record every question that required undocumented author knowledge. If they must create a missing directory, edit an absolute path or guess a data schema, fix the artifact or document the required setup and rerun. A walkthrough with the author clicking all the buttons is not the same test.

Test an expected failure as well: an invalid descriptor, a missing optional dependency or a deliberately changed file. Good error messages should explain what failed and the next safe action. Do not automatically install privileged software, alter firmware or delete user data to make a reproduction script appear effortless.

Plan ninety days around one primary gap

Choose the gap revealed by evidence, not the most fashionable topic. For example, an architecture learner may need stronger model validation; a hardware learner may need actual STA/physical-flow experience; an engine learner may need mature-codebase review. A concrete plan has a 30-day repair target, a 60-day applied artifact and a 90-day reviewed contribution. Allocate a sustainable weekly budget and define what will be dropped if time shrinks.

One primary gap and one real contribution goal are enough. Keep a secondary parking lot rather than starting many unrelated projects. Reassess with an unseen transfer task and a reviewer, then update the plan using evidence. Course completion should increase independence, not create dependence on an endless checklist.

Close Gate J honestly

Gate J closes only when the thesis, independent reproduction, defense, revisions and handoff are complete. The teaching package itself does not close the gate for the learner. A final record names artifacts, evidence tiers, reviewers, dates, unresolved nonblocking limitations and the next contribution. It should state self-study program completion and avoid suggesting an accredited credential.

Before the final lab: remove access to your shell history and personal notes. Check whether the release itself contains enough information for the next maintainer to succeed.

Practical assignment

Complete Gate J: archive the thesis artifact, produce a maintainer handoff and write a 90-day learning plan with one primary gap and one real contribution goal. Pass: another person can locate, build and understand the project without the author's machine.

Use Lab 50.4 in the workbook for procedures, deliverables and acceptance conditions.

Assigned reading

R05: maintenance workflows; R01-R02: periodic breadth review; current official documentation for the selected tools when an update is undertaken.

Expansion sources: S19. See the Math and Reference guide.

Lesson quiz — attempt before opening the answer key

50.4.Q1 Why should a trusted checksum manifest be retained separately from the archive it describes?

50.4.Q2 What distinguishes a maintainer handoff test from an author-led demonstration?

50.4.Q3 What should the 90-day plan prioritize after Gate J?